

Express 演習課題

Express を使った Web アプリケーション開発の段階的な演習課題です。
レベルC → レベルB → レベルA の順に難易度が上がります。

評価基準

レベル	難易度	評価	課題内容
レベルC	★☆☆	C評価	足し算アプリ
レベルB	★★☆	B評価	メモ管理アプリ
レベルA	★★★	A評価	書籍管理アプリ（完全版CRUD）

レベルC: 足し算アプリ（C評価）

目標

2つの数値を入力して、足し算の結果を表示する簡単な Web アプリを作成する。

要件

必須機能

1. 入力画面（GET /）

- 2つの数値を入力できるフォーム
- 「計算する」ボタン

2. 計算処理（POST /calculate）

- フォームから送信された2つの数値を受け取る
- 足し算を実行
- 結果を表示

技術要件

- Express を使用
- EJS テンプレートエンジンを使用（または HTML 直接レスポンスでも可）
- `express.urlencoded()` ミドルウェアでフォームデータを受け取る

実装例の構成

```
level-c/  
├─ server.js      # Express サーバー  
└─ views/
```

```
|   |─ index.ejs      # 入力フォーム  
|   |─ result.ejs    # 計算結果表示  
|   └─ package.json
```

動作イメージ

1. `http://localhost:3000/` にアクセス
2. 「数値1」に `10`、「数値2」に `20` を入力
3. 「計算する」ボタンをクリック
4. 「`10 + 20 = 30`」と表示される

ヒント

```
// フォームデータの受け取り方  
app.post('/calculate', (req, res) => {  
  const num1 = parseInt(req.body.number1);  
  const num2 = parseInt(req.body.number2);  
  const result = num1 + num2;  
  // ...  
});
```

レベルB: メモ管理アプリ（B評価）

目標

メモの追加・一覧表示・削除ができる Web アプリを作成する。

要件

必須機能

1. **メモ一覧表示（GET /）**
 - 保存されているすべてのメモを一覧表示
 - メモがない場合は「メモがありません」と表示
2. **メモ追加（POST /memos）**
 - フォームからメモの内容を受け取る
 - メモを配列に追加（メモリ内保存でOK）
 - 追加後は一覧ページにリダイレクト
3. **メモ削除（POST /memos/delete/:id または DELETE /memos/:id）**
 - 指定されたIDのメモを削除
 - 削除後は一覧ページにリダイレクト

データ構造

```
// メモの例
const memos = [
  { id: 1, content: '買い物に行く', createdAt: '2026-02-23' },
  { id: 2, content: '課題を提出する', createdAt: '2026-02-23' }
];
```

技術要件

- Express を使用
- EJS テンプレートエンジンを使用
- データはメモリ内の配列で管理（再起動すると消える）
- IDは自動採番（配列の長さ + 1 や日時など）

実装例の構成

```
level-b/
├── server.js          # Express サーバー
├── views/
│   └── index.ejs      # メモ一覧 + 追加フォーム
└── package.json
```

動作イメージ

1. <http://localhost:3000/> にアクセス
2. メモ一覧と入力フォームが表示される
3. 「買い物に行く」と入力して「追加」ボタンをクリック
4. ページがリロードされ、追加したメモが一覧に表示される
5. 「削除」ボタンをクリックすると、そのメモが削除される

ヒント

```
// メモの追加
let nextId = 1;
const memos = [];

app.post('/memos', (req, res) => {
  const newMemo = {
    id: nextId++,
    content: req.body.content,
    createdAt: new Date().toISOString().split('T')[0]
  };
  memos.push(newMemo);
  res.redirect('/');
```

```
});

// メモの削除
app.post('/memos/delete/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = memos.findIndex(m => m.id === id);
  if (index !== -1) {
    memos.splice(index, 1);
  }
  res.redirect('/');
});
```

レベルA: 書籍管理アプリ（A評価）

目標

書籍の登録・一覧表示・更新・削除（完全なCRUD操作）ができる Web アプリを作成する。

要件

必須機能

1. 書籍一覧表示（GET /books）

- 登録されているすべての書籍を一覧表示
- 表示項目: 名称、著者、発売年月日、価格
- 各書籍に「編集」「削除」ボタンを表示

2. 書籍詳細表示（GET /books/:id）

- 指定されたIDの書籍の詳細を表示

3. 書籍登録（POST /books）

- フォームから書籍情報を受け取る
- 新しい書籍を配列に追加
- バリデーション: すべての項目が入力されているか確認

4. 書籍更新（PUT /books/:id）

- 指定されたIDの書籍情報を更新
- 更新フォームは GET /books/:id/edit で表示

5. 書籍削除（DELETE /books/:id）

- 指定されたIDの書籍を削除
- 削除前に確認メッセージを表示（JavaScript の confirm）

データ構造

```
// 書籍の例
const books = [
  {
    id: 1,
    title: 'Node.js入門',
    author: '山田太郎',
    publishDate: '2025-01-15',
    price: 2800
  },
  {
    id: 2,
    title: 'Express実践ガイド',
    author: '佐藤花子',
    publishDate: '2025-03-20',
    price: 3200
  }
];
```

技術要件

- Express を使用
- EJS テンプレートエンジンを使用
- REST API の設計に従う（GET, POST, PUT, DELETE を正しく使い分ける）
- データはメモリ内の配列で管理
- **JavaScript の Fetch API を使用して PUT/DELETE リクエストを送信**
- 入力バリデーションを実装
- スタイルシート（CSS）で見た目を整える

実装例の構成

```
level-a/
├── server.js           # Express サーバー
├── routes/
│   └── books.js       # 書籍関連のルーティング（省略可）
├── views/
│   ├── books/
│   │   ├── index.ejs  # 書籍一覧
│   │   ├── new.ejs    # 新規登録フォーム
│   │   ├── edit.ejs   # 編集フォーム
│   │   └── show.ejs   # 詳細表示（省略可）
│   └── layout.ejs     # 共通レイアウト（省略可）
├── public/
│   └── stylesheets/
│       └── style.css   # スタイルシート
└── package.json
```

動作イメージ

1. `http://localhost:3000/books` にアクセス
2. 書籍一覧が表示される
3. 「新規登録」ボタンをクリック → 登録フォームが表示される
4. 書籍情報を入力して「登録」ボタンをクリック
5. 一覧ページにリダイレクトされ、新しい書籍が表示される
6. 「編集」ボタンをクリック → 編集フォームが表示される
7. 情報を変更して「更新」ボタンをクリック
8. 「削除」ボタンをクリック → 確認ダイアログが表示され、OKで削除される

ボーナス課題（余裕があれば）

- ☐ 書籍の検索機能（タイトルや著者で検索）
- ☐ 並び替え機能（価格順、発売日順など）
- ☐ ページネーション（一覧が多い場合の分割表示）
- ☐ localStorage や sessionStorage を使ったデータ永続化
- ☐ 成功・エラーメッセージの表示

ヒント: Fetch API を使った PUT/DELETE の実装方法

HTMLフォームは GET と POST しかサポートしていないため、PUT/DELETE を使うには JavaScript の Fetch API を使用します。

サーバー側 (Express)

```
// 書籍情報を更新
app.put('/books/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = books.findIndex(b => b.id === id);

  if (index !== -1) {
    books[index] = { id, ...req.body };
    res.json(books[index]);
  } else {
    res.status(404).json({ message: 'Book not found' });
  }
});

// 書籍を削除
app.delete('/books/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = books.findIndex(b => b.id === id);

  if (index !== -1) {
    const deleted = books.splice(index, 1)[0];
    res.json(deleted);
  } else {
    res.status(404).json({ message: 'Book not found' });
  }
});
```

クライアント側 (JavaScript)

```
// 書籍情報を更新
async function updateBook(bookId) {
  const formData = {
    title: document.getElementById('title').value,
    author: document.getElementById('author').value,
    publishDate: document.getElementById('publishDate').value,
    price: parseInt(document.getElementById('price').value)
  };

  try {
    const response = await fetch(`/books/${bookId}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(formData)
    });

    if (response.ok) {
      alert('更新しました');
      window.location.href = '/books';
    } else {
      alert('更新に失敗しました');
    }
  } catch (error) {
    console.error('Error:', error);
  }
}

// 書籍を削除
async function deleteBook(bookId) {
  if (!confirm('本当に削除しますか?')) {
    return;
  }

  try {
    const response = await fetch(`/books/${bookId}`, {
      method: 'DELETE'
    });

    if (response.ok) {
      alert('削除しました');
      window.location.href = '/books';
    } else {
      alert('削除に失敗しました');
    }
  } catch (error) {
    console.error('Error:', error);
  }
}
```

```
}  
}
```

HTML側（削除ボタンの例）

```
<button onclick="deleteBook(<%= book.id %>)">削除</button>
```

提出方法

1. 各レベルのフォルダ（`level-c/`, `level-b/`, `level-a/`）を作成
2. 各フォルダ内に実装したコードを配置
3. 各フォルダに `README.md` を作成し、以下を記載:
 - アプリの説明
 - 実装した機能
 - 起動方法
 - 工夫した点や感想

学習のポイント

レベルC で学ぶこと

- Express の基本的な使い方
- ルーティング（GET, POST）
- テンプレートエンジン（EJS）の基礎
- フォームデータの受け取り方

レベルB で学ぶこと

- データの管理（配列操作）
- リダイレクト処理
- パスパラメータの使い方
- 基本的なCRUD操作（Create, Delete）

レベルA で学ぶこと

- 完全なCRUD操作（Create, Read, Update, Delete）
- REST API の設計思想
- バリデーション
- ファイル構成の設計
- ユーザビリティの向上

参考リソース

- [Express 公式ドキュメント](#)

- [EJS 公式ドキュメント](#)
 - [MDN Web Docs - HTML フォーム](#)
-

よくある質問 (FAQ)

Q: レベルCから順番に実装する必要がありますか？

A: はい、推奨します。基礎から段階的に学ぶことで理解が深まります。

Q: データベースを使ってもいいですか？

A: レベルA のボーナス課題として挑戦するのは良いですが、基本課題はメモリ内配列で実装してください。

Q: デザインはどこまで凝る必要がありますか？

A: 最低限見やすければOKです。レベルAでは基本的なCSSを適用することを推奨します。

Q: エラーハンドリングは必要ですか？

A: レベルCとBは最小限でOK。レベルAでは入力バリデーションとエラーメッセージ表示を実装してください。

評価チェックリスト

レベルC

- ☐ Express サーバーが起動する
- ☐ 入力フォームが表示される
- ☐ 2つの数値を入力して計算できる
- ☐ 計算結果が正しく表示される

レベルB

- ☐ メモ一覧が表示される
- ☐ メモを追加できる
- ☐ メモを削除できる
- ☐ メモがない場合の表示がある
- ☐ コードが整理されている

レベルA

- ☐ 書籍一覧が表示される
 - ☐ 書籍を新規登録できる
 - ☐ 書籍情報を更新できる
 - ☐ 書籍を削除できる
 - ☐ すべての項目（名称、著者、発売年月日、価格）が管理できる
 - ☐ バリデーションが実装されている
 - ☐ CSSで見た目が整っている
 - ☐ コードが適切に構造化されている
 - ☐ README.md に起動方法が記載されている
-

頑張ってください！🚀